

Machine Problem 7: Vanilla File System

Nithin Gowtham Saravanan

UIN: 337001954

CSCE611: Operating Systems

Assigned Tasks

Main: Completed.

Bonus Option 1: Design of an extension to the basic file system to allow for files that are up to 64kB long – Completed.

Bonus Option 2: Implementation of the extensions proposed in Option 1– Completed.

Bonus Option 3: Not applicable.

Bonus Option 4: Not applicable.

System Design

The goal of MP7 is to implement a **simple, sequential file system** on top of SimpleDisk, with:

1. A **single-level file namespace** using numerical file IDs (no directories).
2. A **single-block file limit (512B)** in the base implementation.
3. A **bitmap-based free-block list** stored on disk.
4. A clean separation between:
 - **File system metadata & allocation** (FileSystem, Inode)
 - **Sequential access logic** (File)

On top of this vanilla design, an optional **bonus extension** is provided to support files **up to 64kB** using an indexed-allocation scheme. This extension is controlled by a compile-time flag: **#define LARGE_FILE_SUPPORT**

When the macro is disabled, the file system behaves exactly as the vanilla MP7 requires.

High-Level Components

- **SimpleDisk**
 - Provides raw block read/write operations.
 - Unaware of files, inodes, or allocation policies.
 - Serves as the persistence layer for metadata and file data blocks.
- **FileSystem**
 - Manages inodes, free-block allocation, and file creation/deletion.
 - Stores metadata in two fixed disk blocks: **INODES** and **FREELIST**.

- Performs name-to-inode lookup and updates on-disk metadata.
- In bonus mode, manages index blocks for multi-block files.
- **Inode**
 - Holds per-file metadata: ID, length, allocation info, and “used” flag.
 - Vanilla mode: stores a single data block pointer.
 - Large-file mode: stores an index block enabling up to 64kB of data.
- **File**
 - Represents an open sequential file handle with a cached 512B block.
 - Tracks the current position for reads and writes.
 - Handles block loads, writes, and file-length updates.
 - In bonus mode, supports multi-block traversal via logical→physical mapping.
- **Kernel Test Harness**
 - Formats and mounts the file system.
 - Repeatedly creates, writes, reads, and deletes test files.
 - Runs additional 64kB stress tests when large-file support is enabled.

File System Architecture

1. SimpleDisk as the Raw Block Device

- Provides low-level 512-byte block read/write operations through the IDE controller.
- Completely unaware of files, inodes, or allocation.
- All file system operations ultimately map to read(block_no, buf) and write(block_no, buf).

2. FileSystem as Metadata + Allocation Manager

- Owns the inode table, free-block bitmap, and all block allocation logic.
- Performs formatting, mounting, file creation/deletion, and block assignment.
- Disk layout is fixed: **Block 0 = inodes, Block 1 = free list, Blocks 2+ = data blocks.**
- With LARGE_FILE_SUPPORT, also manages index blocks and multi-block allocations.

3. Inode as the File Descriptor

- Represents a single file and encodes its allocation.
- Stores file ID, length, used flag, and either:
 - a single block_no (vanilla), or
 - an index_block_no listing multiple data blocks (large-file mode).
- Acts as the directory entry since file IDs are unique.

4. File as the Sequential Read/Write Interface

- Maintains an in-memory block cache, current position, and (if enabled) current logical block index.

- Reads and writes operate on the cache; disk access happens only on load/close.
- Ensures sequential access semantics and enforces file-length boundaries.

5. Unified Metadata Handling

- Metadata blocks are read, modified in memory, and written back in a consistent pattern.
- FlushMetadata() ensures the inode table and free-block bitmap remain synchronized with disk state.
- No journaling or concurrency is used; simplicity ensures consistency.

6. Indexed Allocation for 64KB Files (Bonus)

- Each file receives a dedicated index block mapping logical → physical data blocks.
- GetDataBlock() handles lookup/allocation; LoadBlock() fetches blocks during I/O.
- FreeAllBlocks() reclaims all data and index blocks during deletion.
- Extends file sizes without changing the external API or kernel logic.

7. Kernel Integration and Testing

- The kernel formats and mounts the file system, then performs repeated correctness tests.
- Baseline tests verify creation, writing, reading, and deletion of one-block files.
- When enabled, large-file tests create two 64KB files, validate block-by-block content, and ensure proper cleanup.
- Macro-guarded implementation allows both modes to run under the same kernel.

Method of Implementation

1. Disk Layout and Metadata

- The file system uses a minimal fixed layout:
Block 0 = INODES, Block 1 = FREELIST, Blocks 2..N = data blocks.
- Both metadata structures are serialized directly to disk and reconstructed on every mount.
- This simple layout satisfies MP7 requirements without directories or superblocks.

2. Inode Structure and File Representation

- Each inode stores: file ID, file length, used flag, and either a single block_no (baseline) or an index_block_no (large-file mode).
- The inode array doubles as a flat directory; lookup is a linear scan.
- Large-file mode uses an index block containing multiple data-block pointers, enabling files up to 64 KB.

3. Free-Block Management

- A byte-array (free_blocks[b]) tracks used/free blocks for simplicity.

- The FREELIST block stores both file system size and block usage, allowing full state reconstruction.
- Allocating or freeing a block updates this array and triggers a metadata flush.

4. Formatting and Mounting

- Format() clears inodes, resets the bitmap, and marks metadata blocks used.
- Mount() reloads the inode table and bitmap from disk and restores inode back-pointers.
- This ensures consistent metadata across executions.

5. File Opening and Caching

- Opening loads the file's first (or only) block into a 512-byte cache and resets the file position.
- All read/write operations occur through this cache.
- File close writes the cache back and flushes metadata.

6. Sequential Read/Write Behavior

- Reads pull bytes from the cache until hitting EOF or requested length.
- Writes update the cache and expand file length, respecting maximum size constraints.
- In large-file mode, reads/writes handle block boundaries, using LoadBlock() for block transitions.

7. Large File Support (Bonus)

- Adds a single-level index block per file, enabling multi-block allocation up to 64 KB.
- GetDataBlock() handles logical-to-physical mapping and allocates blocks on demand.
- FreeAllBlocks() reclaims every block referenced by the index and the index block itself.

8. Kernel Integration and Validation

- The kernel repeatedly tests small-file creation, modification, reading, and deletion.
- With LARGE_FILE_SUPPORT, it additionally runs multi-iteration 64 KB stress tests for two files, verifying integrity.
- Macro-based isolation ensures baseline and bonus paths coexist without changing kernel logic.

CODE Description (baseline implementation)

This implementation modifies the following files:

- file.H
- file.C

- file_system.H
- file_system.C
- macros_config.H
- kernel.C

file.H

1. File state

- fs and inode link a File object to its owning file system and inode.
- current_pos tracks the current read/write offset within the file.

2. Block cache

- block_cache[SimpleDisk::BLOCK_SIZE] holds the file's single data block in memory.
- All reads/writes operate on this cache; the disk is only touched on open/close.

3. Public API behavior

- File(...) initializes the handle at the beginning of the file and loads the block into cache.
- ~File() flushes the cache to disk and updates metadata via the file system.
- Read() copies bytes from the cache up to EOF and advances current_pos.
- Write() writes into the cache up to one block, updates inode->length, and advances current_pos.
- Reset() sets current_pos back to 0; EoF() checks current_pos against inode->length.

4. Macro-guarded extensions

- All large-file-related additions (current_block_index, cache_dirty, LoadBlock()) are wrapped in #ifdef LARGE_FILE_SUPPORT and will be explained separately in the bonus section.

```
private:
    /* -- your file data structures here ... */

    /* You will need a reference to the inode, maybe even a reference to the
       file system.
       You may also want a current position, which indicates which position in
       the file you will read or write next. */

    FileSystem* fs;
    Inode* inode;
    unsigned int current_pos;

#ifdef LARGE_FILE_SUPPORT
    unsigned int current_block_index;
    bool cache_dirty;
#endif

#ifdef LARGE_FILE_SUPPORT
    void LoadBlock(unsigned int _logical_block_index, bool _allocate);
    /* Load the logical block into cache, allocating it if requested. */
#endif
```

file.C – Baseline implementation(Single-Block Files Only)

1. File::File(FileSystem* _fs, int _id)

- Looks up the inode for the given file ID using `_fs->LookupFile(_id)` and stores the result in `inode`.
- Initializes `fs` and sets `current_pos = 0` so the file handle starts at the beginning of the file.
- Asserts that the inode's data block is within the valid file system range (`block_no < fs->GetSizeInBlocks()`).
- Reads the file's single data block from disk into `block_cache` using `fs->GetDisk()->read(...)`, so all subsequent reads/writes operate on the cached copy.

```
File::File(FileSystem* _fs, int _id) {
    Console::puts("Opening file.\n");

    fs = _fs;
    inode = 0;
    current_pos = 0;

    assert(fs != 0);
    inode = fs->LookupFile(_id);
    assert(inode != 0);

    current_pos = 0;
    assert(inode->block_no < fs->GetSizeInBlocks());
    fs->GetDisk()->read(inode->block_no, block_cache);
}
```

2. File::~~File()

- On close, writes the current cached block back to disk using `fs->GetDisk()->write(inode->block_no, block_cache)`, ensuring all in-memory changes are persisted.
- Calls `fs->FlushMetadata()` so the updated inode (e.g., new file length) and free list are saved to the INODES and FREELIST blocks.
- Performs these steps only if both `fs` and `inode` are valid, so invalid/partially constructed objects do not touch the disk.

```
File::~~File() {
    Console::puts("Closing file.\n");
    /* Make sure that you write any cached data to disk. */
    /* Also make sure that the inode in the inode list is updated. */

    if (fs && inode) {
        fs->GetDisk()->write(inode->block_no, block_cache);
    }
    fs->FlushMetadata();
}
```

3. int File::Read(unsigned int _n, char* _buf)

- If current_pos is already at or beyond inode->length, returns 0 immediately (EOF).
- Computes how many bytes can actually be read: to_read = min(_n, inode->length - current_pos).
- Copies to_read bytes from block_cache[current_pos] into _buf.
- Advances current_pos by to_read and returns the number of bytes read.

```
int File::Read(unsigned int _n, char *_buf) {
    Console::puts("reading from file\n");

    if (!inode) {
        return 0;
    }

    if (current_pos >= inode->length) {
        return 0;
    }

    unsigned int remaining = inode->length - current_pos;
    unsigned int to_read = (_n < remaining) ? _n : remaining;

    for (unsigned int i = 0; i < to_read; ++i) {
        _buf[i] = block_cache[current_pos + i];
    }

    current_pos += to_read;
    return static_cast<int>(to_read);
}
```

4. int File::Write(unsigned int _n, const char* _buf)

- Enforces the maximum file size of **one block**: if current_pos is at or beyond SimpleDisk::BLOCK_SIZE, it returns 0.
- Computes available space in the block: remaining_space = SimpleDisk::BLOCK_SIZE - current_pos, then to_write = min(_n, remaining_space).
- Copies to_write bytes from _buf into block_cache[current_pos].
- Advances current_pos and updates inode->length if the file grew past its previous length.
- Returns the number of bytes actually written (never more than one block total).

```
int File::Write(unsigned int _n, const char *_buf) {
    Console::puts("writing to file\n");

    if (!inode) {
        return 0;
    }

    if (current_pos >= SimpleDisk::BLOCK_SIZE) {
        return 0;
    }

    unsigned int remaining_space = SimpleDisk::BLOCK_SIZE - current_pos;
    unsigned int to_write = (_n < remaining_space) ? _n : remaining_space;

    for (unsigned int i = 0; i < to_write; ++i) {
        block_cache[current_pos + i] = _buf[i];
    }

    current_pos += to_write;

    if (current_pos > inode->length) {
        inode->length = current_pos;
    }

    return static_cast<int>(to_write);
}
```

5. void File::Reset()

- Simply sets `current_pos = 0`, so subsequent reads/writes start again from the beginning of the file.
- Does not touch the cache or disk; it only moves the logical file position.

```
void File::Reset() {
    Console::puts("resetting file\n");
    /* Set the 'current position' to be at the beginning of the file. */
    current_pos = 0;
}
```

6. bool File::EoF()

- Returns true if `current_pos >= inode->length`, indicating no more data can be read.
- Returns false otherwise, allowing the caller to detect end-of-file without performing another read.

```
bool File::EoF() {
    Console::puts("checking for EoF\n");
    /* Is the current position for the file at the end of the file? */

    if (!inode) {
        return true;
    }
    return (current_pos >= inode->length);
}
```

file_system.H**1. Core metadata structures**

- An in-memory inode table representing all files in the system.
- Each inode stores: file ID, one data block pointer, file length, and a “used” flag.

2. Free-block management

- A simple bitmap/byte-array (`free_blocks`) tracks which disk blocks are free or used.
- Mirrors the on-disk FREELIST block.

3. File system state

- Stores a pointer to the underlying disk and total number of usable blocks.
- Uses a mounted flag to prevent file operations before initialization.

4. Internal helper routines

- Functions to load/save inode table and free-block list.
- Functions to allocate a free inode or a free data block.
- Small static helpers for manipulating the free-block bitmap.

5. Public interface

- Formatting a fresh file system.
- Mounting an existing file system.
- Creating, looking up, and deleting files.
- Flushing metadata updates back to disk.

6. Macro-guarded bonus extensions

- Large-file fields (`index_block_no`, `MAX_FILE_BLOCKS`) and advanced allocation helpers are wrapped in `#ifdef LARGE_FILE_SUPPORT` and documented separately in the bonus design section.

```
SimpleDisk* disk;
unsigned int size; /* Size of the file system in blocks. */

static constexpr unsigned int MAX_INODES = SimpleDisk::BLOCK_SIZE / sizeof(Inode);
/* Just as an example; you can store MAX_INODES in a single INODES block */

Inode* inodes;
/* The inode list */

unsigned char* free_blocks;

short GetFreeInode();
/* Find a free inode index, or -1 if none is available. */

int GetFreeBlock();
/* Find a free data block index, or -1 if none is available. */

void load_inodes();
/* Read inode list from disk into memory. */

void save_inodes();
/* Write inode list from memory back to disk. */

void load_freelist();
/* Read free-block information from disk into memory. */

void save_freelist();
/* Write free-block information from memory back to disk. */

static void set_block_used(unsigned char* _buf, unsigned int _block_no);
/* Mark a block as used in the on-disk bitmap. */

static void set_block_free(unsigned char* _buf, unsigned int _block_no);
/* Mark a block as free in the on-disk bitmap. */

static bool is_block_used(const unsigned char* _buf, unsigned int _block_no);
/* Check whether a block is used in the on-disk bitmap. */
```

file_system.C

1. FileSystem::FileSystem()

- Initializes all internal pointers (`disk`, `inodes`, `free_blocks`) to null and size to 0.
- Marks the file system as not mounted.
- Ensures that no disk operations or metadata accesses happen before a successful `Mount()`.

```
FileSystem::FileSystem() {
    Console::puts("In file system constructor.\n");

    disk      = 0;
    size      = 0;
    inodes    = 0;
    free_blocks = 0;
    mounted   = false;
}
```

2. FileSystem::~~FileSystem()

- If the file system is mounted and a disk is attached, writes the inode table and free-block list back to disk.
- Frees the in-memory inode array and free-block bitmap.
- Resets internal state so no dangling pointers remain.

```

FileSystem::~FileSystem() {
    Console::puts("unmounting file system\n");
    /* Make sure that the inode list and the free list are saved. */

    if (mounted && disk != 0) {
        save_inodes();
        save_freelist();
    }

    if (inodes) {
        delete[] inodes;
        inodes = 0;
    }
    if (free_blocks) {
        delete[] free_blocks;
        free_blocks = 0;
    }

    disk = 0;
    size = 0;
    mounted = false;
}

```

3. void FileSystem::load_inodes()

- Reads the INODES block from disk into a temporary buffer.
- Copies the raw bytes into the in-memory inodes array.
- Restores each inode's fs pointer to point to this FileSystem instance.

```

void FileSystem::load_inodes() {
    /* Read inode list from disk into memory. */

    unsigned char buf[SimpleDisk::BLOCK_SIZE];
    disk->read(INODES_BLOCK, buf);

    if (!inodes) {
        inodes = new Inode[MAX_INODES];
    }

    unsigned int bytes_to_copy = MAX_INODES * sizeof(Inode);
    if (bytes_to_copy > SimpleDisk::BLOCK_SIZE) {
        bytes_to_copy = SimpleDisk::BLOCK_SIZE;
    }

    for (unsigned int i = 0; i < bytes_to_copy; ++i) {
        reinterpret_cast<unsigned char*>(inodes)[i] = buf[i];
    }

    for (unsigned int i = 0; i < MAX_INODES; ++i) {
        inodes[i].fs = this;
    }
}

```

4. void FileSystem::save_inodes()

- Serializes the in-memory inodes array into a temporary buffer.
- Writes that buffer back to the INODES block on disk.
- Ensures on-disk inode metadata reflects the current in-memory state.

```

void FileSystem::save_inodes() {
    /* Write in-memory inode list back to the INODES block on disk. */

    if (!disk || !inodes) {
        return;
    }

    unsigned char buf[SimpleDisk::BLOCK_SIZE];
    for (unsigned int i = 0; i < SimpleDisk::BLOCK_SIZE; ++i) {
        buf[i] = 0;
    }

    unsigned int bytes_to_copy = MAX_INODES * sizeof(Inode);
    if (bytes_to_copy > SimpleDisk::BLOCK_SIZE) {
        bytes_to_copy = SimpleDisk::BLOCK_SIZE;
    }

    for (unsigned int i = 0; i < bytes_to_copy; ++i) {
        buf[i] = reinterpret_cast<unsigned char*>(inodes)[i];
    }

    disk->write(INODES_BLOCK, buf);
}

```

5. void FileSystem::load_freelist()

- Reads the FREELIST block from disk into a temporary buffer.
- Extracts the stored file system size from the first few bytes.
- Reconstructs the free_blocks array in memory by decoding which bits/entries are marked used vs. free.
- Sets size based on the decoded value.

```
void FileSystem::load_freelist() {
    /* Read and decode free-block list from the FREELIST block. */

    unsigned char buf[SimpleDisk::BLOCK_SIZE];
    disk->read(FREELIST_BLOCK, buf);

    unsigned int fs_blocks = 0;
    fs_blocks |= static_cast<unsigned int>(buf[0]);
    fs_blocks |= static_cast<unsigned int>(buf[1]) << 8;
    fs_blocks |= static_cast<unsigned int>(buf[2]) << 16;
    fs_blocks |= static_cast<unsigned int>(buf[3]) << 24;

    if (fs_blocks == 0) {
        size = 0;
        return;
    }

    size = fs_blocks;

    if (free_blocks) {
        delete[] free_blocks;
    }
    free_blocks = new unsigned char[size];

    for (unsigned int b = 0; b < size; ++b) {
        free_blocks[b] = is_block_used(buf, b) ? 1 : 0;
    }
}
```

6. void FileSystem::save_freelist()

- Encodes the current file system size into the first bytes of a buffer.
- Converts the in-memory free_blocks array into a compact bitmap inside the buffer.
- Writes the buffer to the FREELIST block on disk so allocation state is persisted.

```
void FileSystem::save_freelist() {
    /* Encode and write free-block list into FREELIST block on disk. */

    if (!disk || !free_blocks || size == 0) {
        return;
    }

    unsigned char buf[SimpleDisk::BLOCK_SIZE];
    for (unsigned int i = 0; i < SimpleDisk::BLOCK_SIZE; ++i) {
        buf[i] = 0;
    }

    buf[0] = static_cast<unsigned char>(size >> 24 & 0xFF);
    buf[1] = static_cast<unsigned char>(size >> 16 & 0xFF);
    buf[2] = static_cast<unsigned char>(size >> 8 & 0xFF);
    buf[3] = static_cast<unsigned char>(size & 0xFF);

    for (unsigned int b = 0; b < size; ++b) {
        if (free_blocks[b]) set_block_used(buf, b);
        else set_block_free(buf, b);
    }

    disk->write(FREELIST_BLOCK, buf);
}
```

7. short FileSystem::GetFreeInode()

- Scans through the inode array for an entry whose used flag is false.
- Returns the index of the first such inode, or -1 if none are available.
- Used during file creation to allocate space for a new file's metadata.

```
short FileSystem::GetFreeInode() {
    /* Scan inode list for an unused inode. */

    for (unsigned int i = 0; i < MAX_INODES; ++i) {
        if (!inodes[i].used) {
            return static_cast<short>(i);
        }
    }
    return -1;
}
```

8. int FileSystem::GetFreeBlock()

- Scans the free_blocks array from the first data block onward (skipping metadata blocks).
- Returns the index of the first free data block and marks it as used.
- Returns -1 if no free data blocks remain.

```
int FileSystem::GetFreeBlock() {
    /* Scan for a free data block (skipping metadata blocks). */

    if (!free_blocks || size <= METADATA_BLOCKS) {
        return -1;
    }

    for (unsigned int b = METADATA_BLOCKS; b < size; ++b) {
        if (!free_blocks[b]) {
            free_blocks[b] = 1;
            return static_cast<int>(b);
        }
    }
    return -1;
}
```

9. bool FileSystem::Mount(SimpleDisk* _disk)

- Associates the file system with the given disk.
- Loads the free-block list from the FREELIST block and reconstructs size.
- Verifies that the recorded size does not exceed the physical disk capacity.
- Loads the inode table from the INODES block into memory.
- Marks the file system as mounted and ready for file operations.

```
bool FileSystem::Mount(SimpleDisk * _disk) {
    Console::puts("mounting file system from disk\n");
    /* Here you read the inode list and the free list into memory */

    if (!_disk) {
        return false;
    }

    disk = _disk;

    load_freelist();
    if (size == 0) {
        return false;
    }

    unsigned int disk_blocks = disk->NaiveSize() / SimpleDisk::BLOCK_SIZE;
    assert(size <= disk_blocks);

    load_inodes();

    mounted = true;
    return true;
}
```

10. bool FileSystem::Format(SimpleDisk* _disk, unsigned int _size)

- Computes how many disk blocks the new file system will occupy.
- Initializes a temporary inode array with all entries marked unused and zeroed.
- Initializes a fresh FREELIST buffer, marking only the INODES and FREELIST blocks as used.
- Writes the inode table and freelist to their reserved blocks on disk.
- Produces a clean, empty file system for testing.

```

bool FileSystem::Format(SimpleDisk* _disk, unsigned int _size) { //
static!
    Console::puts("formatting disk\n");
    /* Here you populate the disk with an initialized (probably empty)
inode list
    and a free list. Make sure that blocks used for the inodes and for
the free list
    are marked as used, otherwise they may get overwritten. */
    if (!_disk) {
        return false;
    }

    unsigned int fs_blocks = _size / SimpleDisk::BLOCK_SIZE;
    if (fs_blocks < METADATA_BLOCKS) {
        return false;
    }

    unsigned int max_blocks_encodable = (SimpleDisk::BLOCK_SIZE - 4) * 8;
    assert(fs_blocks <= max_blocks_encodable);

    Inode temp_inodes[FileSystem::MAX_INODES];

    for (unsigned int i = 0; i < FileSystem::MAX_INODES; ++i) {
        temp_inodes[i].id = -1;
        temp_inodes[i].block_no = 0;
        temp_inodes[i].length = 0;
        temp_inodes[i].used = false;
        temp_inodes[i].fs = 0;
    }

    unsigned char inode_buf[SimpleDisk::BLOCK_SIZE];
    for (unsigned int i = 0; i < SimpleDisk::BLOCK_SIZE; ++i) {
        inode_buf[i] = 0;
    }

    unsigned int bytes_to_copy = FileSystem::MAX_INODES * sizeof(Inode);

```

```

•   if (bytes_to_copy > SimpleDisk::BLOCK_SIZE) {
•       bytes_to_copy = SimpleDisk::BLOCK_SIZE;
•   }
•
•   for (unsigned int i = 0; i < bytes_to_copy; ++i) {
•       inode_buf[i] = reinterpret_cast<unsigned char*>(temp_inodes)[i];
•   }
•
•   _disk->write(INODES_BLOCK, inode_buf);
•
•   unsigned char freelist_buf[SimpleDisk::BLOCK_SIZE];
•   for (unsigned int i = 0; i < SimpleDisk::BLOCK_SIZE; ++i) {
•       freelist_buf[i] = 0;
•   }
•
•   freelist_buf[0] = static_cast<unsigned char>(fs_blocks & 0xFF);
•   freelist_buf[1] = static_cast<unsigned char>((fs_blocks >> 8) & 0xFF);
•   freelist_buf[2] = static_cast<unsigned char>((fs_blocks >> 16) &
0xFF);
•   freelist_buf[3] = static_cast<unsigned char>((fs_blocks >> 24) &
0xFF);
•
•   set_block_used(freelist_buf, INODES_BLOCK);
•   set_block_used(freelist_buf, FREELIST_BLOCK);
•
•   _disk->write(FREELIST_BLOCK, freelist_buf);
•
•   return true;
• }

```

11. Inode* FileSystem::LookupFile(int _file_id)

- Performs a linear scan over inodes.
- Returns a pointer to the inode whose id matches _file_id and is marked used.
- Returns nullptr if no such inode exists.

```

Inode * FileSystem::LookupFile(int _file_id) {
    Console::puts("looking up file with id = "); Console::puti(_file_id); Console::puts("\n");
    /* Here you go through the inode list to find the file. */

    if (!inodes) {
        return 0;
    }

    for (unsigned int i = 0; i < MAX_INODES; ++i) {
        if (inodes[i].used && inodes[i].id == _file_id) {
            return &inodes[i];
        }
    }

    return 0;
}

```

12. bool FileSystem::CreateFile(int _file_id)

- Rejects creation if LookupFile(_file_id) finds an existing file.
- Allocates a free inode via GetFreeInode() and a free data block via GetFreeBlock().
- Initializes the inode with the file ID, the allocated block number, zero length, and used = true.
- Zeros the newly allocated data block on disk.
- Saves the updated inode table and free-block list to disk.

```

• bool FileSystem::CreateFile(int _file_id) {
•     Console::puts("creating file with id:"); Console::puti(_file_id);
•     Console::puts("\n");
•     if (!mounted || !disk) {
•         return false;
•     }
•     if (LookupFile(_file_id) != 0) {
•         return false;
•     }
•     short inode_index = GetFreeInode();
•     if (inode_index < 0) {
•         return false;
•     }
•     Inode& inode = inodes[inode_index];
•
•     int block_index = GetFreeBlock();
•     if (block_index < 0) {
•         return false;
•     }
•     inode.id      = static_cast<long>(_file_id);
•     inode.block_no = static_cast<unsigned int>(block_index);
•     inode.length  = 0;
•     inode.used    = true;
•     inode.fs      = this;
•
•     unsigned char zero_block[SimpleDisk::BLOCK_SIZE];
•     for (unsigned int i = 0; i < SimpleDisk::BLOCK_SIZE; ++i) {
•         zero_block[i] = 0;
•     }
•     disk->write(inode.block_no, zero_block);
•     save_inodes();
•     save_freelist();
•
•     return true;
• }

```

13. bool FileSystem::DeleteFile(int _file_id)

- Locates the inode using LookupFile(_file_id).
- If found, frees the file's single data block in free_blocks.
- Resets the inode fields (invalid ID, zero length, used = false).
- Writes back the inode table and free-block list to disk.

```
bool FileSystem::DeleteFile(int _file_id) {
    Console::puts("deleting file with id:"); Console::puti(_file_id);
    Console::puts("\n");
    if (!mounted || !disk) {
        return false;
    }
    Inode* inode = LookupFile(_file_id);
    if (!inode) {
        return false;
    }
    if (inode->block_no < size && inode->block_no >= METADATA_BLOCKS &&
        free_blocks) {
        free_blocks[inode->block_no] = 0;
    }
    inode->id      = -1;
    inode->length  = 0;
    inode->used    = false;
    save_inodes();
    save_freelist();
    return true;
}
```

14. void FileSystem::FlushMetadata()

- Convenience function that calls save_inodes() and save_freelist().
- Used by higher-level components (like File::~File()) to persist structural changes after file operations.

```
void FileSystem::FlushMetadata() {
    /* Helper to flush inode and free-block metadata back to disk. */
    if (!disk) {
        return;
    }
    save_inodes();
    save_freelist();
}
```

macros_config.H

To ensure backwards compatibility between bonus options

```
#define LARGE_FILE_SUPPORT // uncomment to handle large files
```


kernel.C

1.Large-File Stress Test Loop (IDs 10 and 11)

- A new block of code inside `#ifdef LARGE_FILE_SUPPORT` creates two large files, writes 64 kB patterned data into them, reopens them, verifies full multi-block correctness, and deletes them—repeated for 30 iterations.
- This code includes new buffers, write/read validation, and detailed logging specific to large-file behavior.

2.Pattern Initialization for Large Test Buffers

- Additional logic initializes two 64 kB write buffers with iteration-dependent byte patterns, ensuring that every large-file test iteration uses distinct, verifiable data.
- Matching read buffers are compared byte-for-byte to confirm correct multi-block reads and writes.

```
#ifdef LARGE_FILE_SUPPORT
/* -- BONUS TEST: Stress-test large file support with two files for 30 iterations -- */
for (int iter = 0; iter < 30; iter++) {
    Console::puts("exercise LARGE file system; iteration ");
    Console::puti(iter);
    Console::puts("...\n");
}
```

Please refer to the kernel.c file for further implementation

Bonus Option 1 - Design of a large file system

Core Allocation Strategy

The baseline file system limits each file to a single 512-byte block. To support files up to 64 kB, the design introduces a single-level indexed allocation scheme: each file is given an index block that stores an array of 4-byte disk block numbers. A 512-byte index block can hold 128 entries, so a file can span up to 128 data blocks, for a total of 128×512 bytes = 64 kB. This replaces the single data block pointer with a compact in-file allocation table while preserving the simple global layout of metadata and data blocks.

Metadata and FileSystem Changes

The inode structure is modified by replacing the original `block_no` field with `index_block_no`, which points to the dedicated index block of that file. A constant `MAX_FILE_BLOCKS = BLOCK_SIZE / sizeof(uint32_t)` defines the maximum number of data blocks a file may use. The FileSystem is extended with a helper `GetDataBlock(inode, logical_index, allocate)` that reads the index block, returns the physical block number for a given logical index, and optionally allocates a new data block if the entry is empty. Another helper,

FreeAllBlocks(inode), iterates over the index block to free all referenced data blocks and then frees the index block itself during file deletion. CreateFile() now allocates and zeroes an index block per file, and DeleteFile() delegates reclamation to FreeAllBlocks(), while the on-disk layout of the INODES and FREELIST blocks remains unchanged.

Sequential Access and File Behavior

The File class is extended with a current logical block index and a dirty flag for its 512-byte cache. A new method LoadBlock(logical_index, allocate) flushes the cache if dirty, uses GetDataBlock() to find or allocate the corresponding physical block, and loads its contents into memory. Read operations compute the logical block index and offset from the current file position and may call LoadBlock() multiple times as they cross block boundaries, stopping when either the requested byte count is satisfied or the file length is reached. Write operations follow the same pattern but allow LoadBlock() to allocate new blocks when necessary, growing the file length up to the 64 kB limit. When the File object is destroyed, any dirty cached data is written back to disk and metadata is flushed to keep the inode and free-block list consistent.

Summary

The extension therefore adds a single index block per file, a small amount of allocation logic in the FileSystem, and multi-block awareness in File's read and write paths. The disk layout, single-level "directory" model, and public APIs remain the same, and the entire design can be cleanly enabled or disabled through a compile-time flag without affecting the baseline one-block implementation.

Bonus Option 2 - Implementation of a large file system

- **FileSystem Modifications (file_system.C)**

The inode now stores an index_block_no, allocated during CreateFile() and initialized to a zeroed 512-byte block. The function GetDataBlock() reads the index block, returns the physical block for a requested logical index, and allocates a new data block when needed. FreeAllBlocks() scans the index block to free all referenced blocks and releases the index block during DeleteFile(). The free-block bitmap logic remains unchanged; only additional blocks are marked used or free as needed. All new logic is guarded by #ifdef LARGE_FILE_SUPPORT, ensuring binary compatibility with the baseline MP7 implementation.

In CreateFile()

```

#ifdef LARGE_FILE_SUPPORT
    int index_block = GetFreeBlock();
    if (index_block < 0) {
        return false;
    }

    inode.id          = static_cast<long>(_file_id);
    inode.index_block_no = static_cast<unsigned int>(index_block);
    inode.length       = 0;
    inode.used         = true;
    inode.fs           = this;

    unsigned char index_buf[SimpleDisk::BLOCK_SIZE];
    for (unsigned int i = 0; i < SimpleDisk::BLOCK_SIZE; ++i) {
        index_buf[i] = 0;
    }
    disk->write(inode.index_block_no, index_buf);
#else

```

In DeleteFile()

```

#ifdef LARGE_FILE_SUPPORT
    FreeAllBlocks(inode);
    inode->index_block_no = 0;
#else
    if (inode->block_no < size && inode->block_no >= METADATA_BLOCKS && free_blocks) {
        free_blocks[inode->block_no] = 0;
    }
#endif

```

- **File Class Modifications (file.C)**

The File class introduces a `current_block_index` field, a `cache_dirty` flag, and a helper function `LoadBlock()` that loads the appropriate logical block into the cache, allocating it if required. Reads and writes now compute the logical block index and offset from the current file position and may span multiple blocks through repeated calls to `LoadBlock()`. Writes grow the file length as new blocks are allocated up to the 64 kB limit. On file close, dirty cache contents are flushed to their physical block, and metadata is written back through `FlushMetadata()`.

In constructor()

```

#ifdef LARGE_FILE_SUPPORT
    current_block_index = 0;
    cache_dirty         = false;

    if (inode->length > 0) {
        LoadBlock(0, false);
    } else {
        for (unsigned int i = 0; i < SimpleDisk::BLOCK_SIZE; ++i) {
            block_cache[i] = 0;
        }
    }
#else

```

In destructor()

```
#ifdef LARGE_FILE_SUPPORT
    if (cache_dirty) {
        unsigned int phys = fs->GetDataBlock(inode, current_block_index, false);
        if (phys != 0) {
            fs->GetDisk()->write(phys, block_cache);
        }
    }
#else
    fs->GetDisk()->write(inode->block_no, block_cache);
#endif
```

LoadBlock()

```
#ifdef LARGE_FILE_SUPPORT
void File::LoadBlock(unsigned int _logical_block_index, bool _allocate) {
    /* Load the logical block into cache, allocating it if requested. */

    if (!inode) {
        return;
    }

    if (cache_dirty) {
        unsigned int old_phys = fs->GetDataBlock(inode, current_block_index, false);
        if (old_phys != 0) {
            fs->GetDisk()->write(old_phys, block_cache);
        }
        cache_dirty = false;
    }

    unsigned int phys = fs->GetDataBlock(inode, _logical_block_index, _allocate);

    if (phys != 0) {
        assert(phys < fs->GetSizeInBlocks());
        fs->GetDisk()->read(phys, block_cache);
    } else {
        for (unsigned int i = 0; i < SimpleDisk::BLOCK_SIZE; ++i) {
            block_cache[i] = 0;
        }
    }

    current_block_index = _logical_block_index;
}
#endif
```

Read the file using the instructions below:

```
if (current_pos >= inode->length) {
    return 0;
}
unsigned int end_pos = current_pos + _n;
if (end_pos > inode->length) {
    end_pos = inode->length;
}
unsigned int total_copied = 0;
while (current_pos < end_pos) {
    unsigned int logical_block_index = current_pos / SimpleDisk::BLOCK_SIZE;
    unsigned int offset_in_block = current_pos % SimpleDisk::BLOCK_SIZE;
    unsigned int remaining = end_pos - current_pos;
```

```

    unsigned int space_in_block    = SimpleDisk::BLOCK_SIZE - offset_in_block;
    unsigned int bytes_in_block    = (remaining < space_in_block) ? remaining : space_in_block;
    LoadBlock(logical_block_index, false);
    for (unsigned int i = 0; i < bytes_in_block; ++i) {
        _buf[total_copied + i] = block_cache[offset_in_block + i];
    }
    current_pos += bytes_in_block;
    total_copied += bytes_in_block;
}
return static_cast<int>(total_copied);

```

Write the file using the instructions below:

```

const unsigned int MAX_FILE_SIZE = Inode::MAX_FILE_BLOCKS * SimpleDisk::BLOCK_SIZE;
if (current_pos >= MAX_FILE_SIZE) {
    return 0;
}
unsigned int max_bytes    = MAX_FILE_SIZE - current_pos;
unsigned int to_write_total = (_n < max_bytes) ? _n : max_bytes;
unsigned int total_written = 0;
while (total_written < to_write_total) {
    unsigned int logical_block_index = current_pos / SimpleDisk::BLOCK_SIZE;
    unsigned int offset_in_block    = current_pos % SimpleDisk::BLOCK_SIZE;
    unsigned int remaining          = to_write_total - total_written;
    unsigned int space_in_block    = SimpleDisk::BLOCK_SIZE - offset_in_block;
    unsigned int bytes_in_block    = (remaining < space_in_block) ? remaining : space_in_block;
    LoadBlock(logical_block_index, true);
    for (unsigned int i = 0; i < bytes_in_block; ++i) {
        block_cache[offset_in_block + i] = _buf[total_written + i];
    }
    cache_dirty = true;
    current_pos += bytes_in_block;
    total_written += bytes_in_block;

    if (current_pos > inode->length) {
        inode->length = current_pos;
    }
}
return static_cast<int>(total_written);

```

- **Kernel Integration**

The kernel includes an additional test loop under `#ifdef LARGE_FILE_SUPPORT` that creates, writes, reads, validates, and deletes two 64 kB files across multiple iterations. This verifies correct allocation across many blocks and ensures the correctness of logical-to-physical mapping, block allocation, and cleanup. The original small-file tests remain unchanged and continue to execute normally.

TESTING

A. Baseline File-System Testing (Small Files)

- The kernel formats and mounts the file system, then runs 30 iterations of creating two files (IDs 1 and 2), writing 20-byte strings to each, and closing them.
- It reopens both files, resets their read positions, and reads back the stored data to verify that the exact strings were preserved.
- After successful verification, both files are deleted and their blocks returned to the free-block list.
- Repeating this process stresses allocation, persistence, caching, and metadata updates while ensuring correctness across many cycles.

```
12 //define LARGE_FILE_SUPPORT // uncommnt to support large files

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

bash - MP7_Sources + ▢ ▢ ... | ↻ ✕

looking up file with id = 1
Opening file.
looking up file with id = 2
Checking content of File 1 and File 2
resetting file
reading from file
resetting file
reading from file
SUCCESS!!
Closing File 1 and File 2 again
Closing file.
Closing file.
Deleting File 1 and File 2
deleting file with id:1
looking up file with id = 1
looking up file with id = 1
deleting file with id:2
looking up file with id = 2
looking up file with id = 2
iteration done
EXCELLENT! Your File system seems to work correctly. Congratulations!!
Assertion failed at file: kernel.C line: 386 assertion: false
One second has passed
One second has passed
qemu: terminating on signal 2

nithin@tham250Nithin:~/MachineProblems OS/MachineProblems git repo/MP7_Sources$
```

Note: A detailed log is stored at kernel_log_baseline.txt. This is executed with the command `make run > kernel_log_baseline.txt 2>&1`

B. Large-File Testing (Requires LARGE_FILE_SUPPORT Enabled)

1. Large-file validation is activated only when the macro `LARGE_FILE_SUPPORT` is enabled in **macros_config.H**, ensuring the baseline system remains untouched by default.

2. With the macro active, the kernel conducts 30 iterations of creating two large files (IDs 10 and 11), each capable of holding 64 kB of data through indexed allocation.
3. In every iteration, the kernel writes a full 64 kB patterned buffer to both files, reopens them, reads back the data across all logical blocks, and performs exact byte-for-byte verification.
4. After successful validation, both large files are deleted, and all associated data blocks—including index blocks—are reclaimed, thoroughly exercising allocation, multi-block access, and cleanup logic.
5. `exercise_file_system()` is also executed to ensure that enabling large file system logic handles the small file systems too.

```

12 #define LARGE_FILE_SUPPORT // uncomment to handle large files

resetting file
resetting file
reading from file
reading from file
Bytes read from Big File 1: 65536
Bytes read from Big File 2: 65536
SUCCESS for large files!!
Closing Big File 1 and Big File 2 again
Closing file.
Closing file.
Deleting Big File 1 and Big File 2
deleting file with id:10
looking up file with id = 10
looking up file with id = 10
deleting file with id:11
looking up file with id = 11
looking up file with id = 11
Iteration done for large file
EXCELLENT! Your File system seems to work correctly. Congratulations!!
Assertion failed at file: kernel.c line: 300 assertion: false
One second has passed
One second has passed
One second has passed
One second has passed
INTERUPT NO: 1
NO DEFAULT INTERRUPT HANDLER REGISTERED

```

6. **Note: A detailed log is stored at `kernel_log_large_files.txt`. This is executed with the command `make run > kernel_log_large_files.txt 2>&1`**

Note: The assertion at the end is voluntary for both cases. Overall, the test was validated for both short and large files.